



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی

بخش مهندسی کامپیوتر

گزارش پروژه کارشناسی

عنوان:

اتوماسیون کشاورزی و سیستم تشخیص آفات و بیماری های

گیاهی مبتنی بر هوش مصنوعی

نگارش:

محمد یاسین بلوچی رضاآبادی

استاد راهنما:

دکتر عباس بحر العلوم

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی
بخش مهندسی کامپیوتر

تاییدیه هیأت داوران

هیأت داوران پس از مطالعه گزارش با عنوان اتوماسیون کشاورزی و سیستم تشخیص آفات و بیماری های گیاهی مبتنی بر هوش به نگارش محمد یاسین بلوچی رضاآبادی ، پروژه انجام شده را برای اخذ درجه **کارشناسی رشته مهندسی کامپیوتر** گرایش هوش مصنوعی مورد تأیید قرار دادند.

امضاء

استاد راهنما: دکتر عباس بحرالعلوم

امضاء

استاد داور: دکتر نام کامل استاد داور



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی
بخش مهندسی کامپیوتر

اظهارنامه دانشجو

اینجانب **محمد یاسین بلوچی** دانشجوی دوره **کارشناسی رشته مهندسی**

کامپیوتر گرایش هوش مصنوعی دانشکده **فنی و مهندسی** دانشگاه **شهید**

باهنر کرمان گواهی می‌نمایم که پروژه ارائه شده در این گزارش با عنوان:

اتوماسیون کشاورزی و سیستم تشخیص آفات و بیماری‌های گیاهی

مبتنی بر هوش مصنوعی

با راهنمایی استاد محترم **جناب آقای دکتر عباس بحر العلوم** توسط شخص

اینجانب انجام شده است. صحت و اصالت مطالب نگارش شده در این پروژه

مورد تأیید اینجانب می‌باشد. در مورد استفاده از کار سایر محققین به مرجع مورد

استفاده اشاره شده است. همچنین، گواهی می‌نمایم که مطالب مندرج در

گزارش پروژه تاکنون برای دریافت هیچ نوع مدرک یا امتیازی توسط اینجانب یا

فرد ديگري در هيچ جا ارائه نشده است و در تدوين متن پروژه چارچوب مصوب
دانشگاه به طور كامل رعايت شده است.

امضاء دانشجو:

تاريخ:



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی
بخش مهندسی کامپیوتر

حق طبع، نشر و مالکیت نتایج

1. حق چاپ و تکثیر این اثر متعلق به نویسنده و استاد راهنمای آن می‌باشد. هرگونه تصویربرداری از کل یا بخشی از آن تنها با موافقت استاد راهنما مجاز می‌باشد.
2. کلیه حقوق معنوی این اثر متعلق به دانشگاه شهید باهنر کرمان می‌باشد و بدون اجازه کتبی دانشگاه به شخص ثالث قابل واگذاری نیست.
3. استفاده از اطلاعات و نتایج موجود در این اثر بدون ذکر مرجع مجاز نمی‌باشد.

تقدیم

این پروژه را تقدیم میکنم به ارواح پاک تمامی شهدای جنگ رمضان که بدست پست
ترین انسان های روی زمین به شهادت رسیدند ، به خصوص رهبر عزیزم و کودکان
مدرسه شجره طیبه میناب .

تشکر و قدردانی

تقدیر و تشکر میکنم از جناب آقای دکتر عباس بهارالعلوم که من را در انجام این پروژه کمک و راهنمایی نموده اند .

چکیده

در دهه‌های اخیر، کشاورزی دقیق و هوشمند به عنوان یکی از مهم‌ترین حوزه‌های کاربردی هوش مصنوعی مطرح شده است. این پروژه با هدف طراحی و پیاده‌سازی یک سیستم یکپارچه اتوماسیون کشاورزی مبتنی بر هوش مصنوعی انجام شده است که قابلیت تشخیص خودکار آفات و بیماری‌های گیاهی را از روی تصویر برگ گیاه فراهم می‌کند.

در این پروژه، مدل یادگیری عمیق MobileNetV2 با استفاده از تکنیک یادگیری انتقالی (Transfer Learning) بر روی مجموعه داده‌های ترکیبی شامل PlantDoc Plant Village، مجموعه داده انگور و گوجه‌فرنگی آموزش دیده است. مجموعه داده نهایی شامل ۴۱.۴۷۴ تصویر از ۲۴ کلاس بیماری و سالم می‌باشد که پس از متوازن‌سازی به ۵.۷۵۴ نمونه آموزش، ۱.۲۳۳ نمونه اعتبارسنجی و ۱.۲۳۳ نمونه آزمایش تقسیم شد.

فرآیند آموزش در سه مرحله انجام شد: ابتدا لایه‌های پایه MobileNetV2 فریز شده و فقط لایه‌های طبقه‌بند جدید آموزش دیدند. سپس در مرحله fine-tuning، آخرین ۳۰ لایه شبکه پایه آزاد شده و با نرخ یادگیری کمتر آموزش دیدند. نتایج نهایی نشان‌دهنده دقت ۸۹.۷۸٪ بر روی مجموعه اعتبارسنجی است.

سیستم شامل سه بخش اصلی است: (۱) وب‌اپلیکیشن React برای مدیریت مزرعه و نمایش داشبورد، (۲) اپلیکیشن موبایل Flutter برای تشخیص میدانی آفات با دوربین گوشی، و (۳) مدل هوش مصنوعی TFLite بهینه‌شده برای اجرا روی دستگاه‌های لبه. این سیستم یکپارچه به کشاورزان امکان می‌دهد تا به‌صورت آنی و با دقت بالا وضعیت سلامت گیاهان خود را ارزیابی کنند.

واژه‌های کلیدی: یادگیری عمیق، تشخیص بیماری گیاهی، MobileNetV2، یادگیری

انتقالی، اتوماسیون کشاورزی، پردازش تصویر، React، Flutter

فهرست مطالب

عنوان	صفحه
فهرست شکلهای.....	د
فهرست جدولها.....	د
فصل اول: مقدمه.....	1
۱-۱- بیان مسئله.....	1
۱-۲- اهداف پروژه.....	1
۱-۳- اهمیت و ضرورت پروژه.....	1
۱-۴- نوآوریهای پروژه.....	2
۱-۵- ساختار گزارش.....	2
فصل دوم: مبانی نظری و مرور ادبیات.....	3
۲-۱- شبکههای عصبی کانولوشنی.....	3
۲-۱-۱- لایه کانولوشن.....	3
۲-۱-۲- لایه Pooling.....	3
۲-۱-۳- لایه Fully Connected.....	3
۲-۲- یادگیری انتقالی.....	3
۲-۳- معماری MobileNetV2.....	4
۲-۴-۲- PlantDoc.....	4
۳-۴-۲- Grape Disease Dataset.....	5

- ۵-۲- روش‌های متوازن‌سازی داده.....5
- ۶-۲- پردازش تصویر و افزایش داده.....5
- ۷-۲- متریک‌های ارزیابی.....5
- فصل سوم: معماری و طراحی سیستم.....7
- ۱-۳- دیدگاه کلی سیستم.....7
- ۲-۳- معماری مدل هوش مصنوعی.....7
- ۳-۲-۲- استراتژی آموزش سه مرحله‌ای.....8
- ۳-۳- معماری وب‌اپلیکیشن.....8
- ۳-۳-۱- داشبورد مدیریت مزرعه.....8
- ۳-۳-۲- سیستم گزارش‌گیری.....8
- ۴-۳- معماری اپلیکیشن موبایل Flutter.....9
- ۳-۴-۱- مازول تشخیص بیماری.....9
- ۳-۴-۲- مدیریت حالت آفلاین.....9
- ۵-۳- جریان داده در سیستم.....9
- فصل چهارم: پیاده‌سازی.....10
- ۱-۴- محیط پیاده‌سازی.....10
- ۲-۴- آماده‌سازی مجموعه داده.....10
- ۴-۲-۱- بارگذاری داده‌ها.....10

۲-۲-۲- ادغام مجموعه داده‌ها	11
۳-۲-۲- متوازن سازی داده	11
۴-۳- طراحی و آموزش مدل	11
۱-۳-۲- ساخت مدل پایه	11
۲-۳-۲- مرحله اول آموزش	12
۳-۳-۲- مرحله دوم: Fine-tuning	12
۴-۳-۲- مرحله سوم: Fine-tuning نهایی	12
۴-۴- بهینه سازی و تبدیل به TFLite	13
۴-۵- فرآیند آموزش مدل و بهینه سازی روی دیتاست واقعی (Fine-tuning)	13
۴-۵-۱- مرحله آموزش اولیه	13
۴-۵-۳- جزئیات فنی فرآیند آموزش (Technical Details)	14
۴-۵-۴- تحلیل نتایج بر روی داده‌های واقعی	15
۴-۵-۵- تنظیم ابرپارامترها و استراتژی بهینه سازی	15
۴-۶- معماری سامانه و پنل‌های کاربری (System Architecture & User Panels)	16
۴-۶-۱- هسته مرکزی و زیرساخت (Core Infrastructure)	16
۴-۶-۲- پنل کشاورز (Farmer Dashboard)	16
۴-۶-۳- پنل خریدار (Buyer Dashboard)	16

۴-۶-۲.	پنل خدمات (Services Dashboard)	16
۴-۶-۵.	ارتباطات و یکپارچگی (Communication & Integration)	17
۴-۷.	پیاده‌سازی اپلیکیشن Flutter	20
۴-۷-۱.	فرآیند استنتاج محلی (Local Inference Pipeline)	20
۴-۷-۲.	ویژگی‌های فنی برتر سیستم موبایل	21
23	فصل پنجم: نتایج و ارزیابی	23
۵-۱.	نتایج آموزش مدل	23
۵-۲.	ارزیابی روی مجموعه آزمایش	23
۵-۳.	تحلیل خطاهای مدل	26
۵-۴.	مقایسه با مدل‌های قبلی	26
۵-۵.	آزمایش اپلیکیشن موبایل	26
۵-۶.	ارزیابی وب‌اپلیکیشن	27
28	نتیجه‌گیری و پیشنهادات	28
28	دستاوردهای اصلی	28
28	پیشنهادهای برای کارهای آینده	28
29	پیوست 1: کدهای اصلی پیاده‌سازی شده	29
29	کد ادغام مجموعه داده‌ها	29
29	کد ساخت مدل MobileNetV2	29

30.....	منابع
31.....	واژه‌نامه فارسی به انگلیسی

فهرست شکلها

عنوان	صفحه
3-4-آموزش و تست مدل.....	14
5-4-پیاده سازی وب اپلیکیشن React.....	16
6-4-پیاده سازی اپلیکیشن Flutter.....	19
2-5-ارزیابی روی مجموعه آزمایش.....	22

فهرست جدولها

عنوان	صفحه
1-2-مقایسه معماری های مختلف CNN.....	4
2-2-متریک های ارزیابی مدل.....	6
1-3-اجزای اصلی سیستم.....	8
1-4-محیط پیاده سازی.....	12
2-4-آمار مجموعه داده نهایی.....	13
1-5-نتایج آموزش در هر مرحله.....	16
2-5-نتایج ارزیابی روی مجموعه آزمایش (برخی کلاس ها).....	17
3-5-مقایسه با کارهای مرتبط.....	18
3-5-عملکرد اپلیکیشن روی دستگاه های مختلف.....	18

فصل اول: مقدمه

۱-۱- بیان مسئله

کشاورزی به عنوان پایه‌ای‌ترین فعالیت اقتصادی بشر، همواره با چالش‌های متعددی روبرو بوده است. یکی از مهم‌ترین این چالش‌ها، آفات و بیماری‌های گیاهی است که هر ساله موجب خسارات سنگین اقتصادی به کشاورزان سراسر جهان می‌شود. بنا بر آمار سازمان خواروبار جهانی (FAO)، بین ۲۰ تا ۴۰ درصد از محصولات کشاورزی جهان به دلیل آفات و بیماری‌ها از بین می‌روند [1].

تشخیص سنتی بیماری‌های گیاهی وابسته به تخصص کارشناسان کشاورزی است که هم پرهزینه است و هم در مناطق دورافتاده در دسترس نیست. این ضرورت، توسعه سیستم‌های هوشمند و خودکار تشخیص بیماری را اجتناب‌ناپذیر ساخته است.

با ظهور یادگیری عمیق و به‌ویژه شبکه‌های عصبی کانولوشنی (CNN)، تشخیص خودکار بیماری‌های گیاهی از روی تصاویر دیجیتال امکان‌پذیر شده است. این رویکرد نه تنها دقت بالایی دارد بلکه می‌تواند در زمان واقعی و با استفاده از دستگاه‌های موبایل در میدان کشاورزی پیاده‌سازی شود.

۱-۲- اهداف پروژه

اهداف اصلی این پروژه به شرح زیر است:

- طراحی و پیاده‌سازی مدل یادگیری عمیق برای تشخیص آفات و بیماری‌های گیاهی با دقت بالا
- توسعه اپلیکیشن موبایل Flutter برای تشخیص میدانی بیماری‌های گیاهی
- ایجاد وب‌اپلیکیشن مدیریت مزرعه با داشبورد جامع
- بهینه‌سازی مدل برای اجرا روی دستگاه‌های لبه با منابع محاسباتی محدود
- یکپارچه‌سازی سه بخش مدل هوش مصنوعی، اپلیکیشن موبایل و وب‌اپلیکیشن

۱-۳- اهمیت و ضرورت پروژه

ایران به عنوان یک کشور کشاورزی با اقلیم متنوع، با طیف گسترده‌ای از آفات و بیماری‌های گیاهی روبرو است. کمبود کارشناسان متخصص در مناطق روستایی،

هزینه بالای مشاوره، و تأخیر در تشخیص همه از عواملی هستند که خسارات کشاورزی را افزایش می‌دهند.

سیستم پیشنهادی در این پروژه با ارائه ابزاری در دسترس و دقیق، می‌تواند نقش مهمی در کاهش این خسارات داشته باشد. کشاورز می‌تواند صرفاً با عکس گرفتن از برگ گیاه با گوشی موبایل، در کمتر از یک ثانیه نتیجه تشخیص بیماری را دریافت کند.

۱-۴- نوآوری‌های پروژه

این پروژه چندین نوآوری مهم دارد:

- ترکیب چندین مجموعه داده عمومی (PlantDoc، PlantVillage، Grape Disease، Tomato) برای ایجاد یک دیتاست جامع‌تر با ۴۱,۴۷۴ تصویر
- استراتژی متوازن‌سازی هوشمند داده با ترکیب undersampling و oversampling
- Fine-tuning سه مرحله‌ای MobileNetV2 با باز کردن تدریجی لایه‌ها
- بهینه‌سازی مدل به فرمت TFLite برای اجرای آفلاین روی موبایل
- معماری یکپارچه شامل موبایل، وب و هوش مصنوعی

۱-۵- ساختار گزارش

این گزارش در پنج فصل تنظیم شده است. فصل دوم به مرور ادبیات و مبانی نظری می‌پردازد. فصل سوم معماری و طراحی سیستم را شرح می‌دهد. فصل چهارم به پیاده‌سازی و جزئیات فنی اختصاص دارد. فصل پنجم نتایج آزمایش‌ها را ارائه می‌دهد و فصل ششم نتیجه‌گیری و پیشنهادات را دربرمی‌گیرد.

فصل دوم: مبانی نظری و مرور ادبیات

۲-۱- شبکه‌های عصبی کانولوشنی

شبکه‌های عصبی کانولوشنی (Convolutional Neural Networks یا CNN) نوعی از شبکه‌های عصبی عمیق هستند که برای پردازش داده‌های شبکه‌ای مانند تصاویر طراحی شده‌اند [2]. این شبکه‌ها از سه نوع لایه اصلی تشکیل شده‌اند:

2-1-1 لایه کانولوشن

لایه کانولوشن اصلی‌ترین بلوک سازنده CNN است. این لایه با اعمال فیلترهای یادگیرفتنی (kernels) بر روی تصویر ورودی، ویژگی‌های مکانی را استخراج می‌کند. هر فیلتر یک ویژگی خاص مانند لبه، بافت یا الگو را شناسایی می‌کند. خروجی این لایه feature map نامیده می‌شود.

2-1-2 لایه Pooling

لایه Pooling با کاهش ابعاد feature map، تعداد پارامترها را کم می‌کند و مقاومت در برابر جابجایی مکانی را افزایش می‌دهد. رایج‌ترین نوع آن Max Pooling است که بیشینه مقدار را در هر پنجره انتخاب می‌کند.

2-1-3 Fully Connected لایه

لایه‌های Fully Connected در انتهای شبکه قرار گرفته و وظیفه طبقه‌بندی بر اساس ویژگی‌های استخراج شده را بر عهده دارند. در این پروژه، دو لایه Dense با ۲۵۶ و ۲۴ نورون (برابر تعداد کلاس‌ها) استفاده شده است.

۲-۲- یادگیری انتقالی

یادگیری انتقالی (Transfer Learning) رویکردی است که در آن دانش کسب‌شده از حل یک مسئله، برای حل مسئله مرتبط دیگری به کار برده می‌شود [3]. در حوزه یادگیری عمیق، این به معنای استفاده از مدل از پیش آموزش‌دیده روی یک مجموعه داده بزرگ (مانند ImageNet) به عنوان نقطه شروع برای آموزش روی مسئله جدید است.

مزایای اصلی این رویکرد شامل نیاز به داده کمتر، زمان آموزش کوتاه‌تر و عموماً دقت بهتر نسبت به آموزش از صفر است. این ویژگی‌ها برای پروژه‌های با محدودیت داده مانند تشخیص بیماری گیاهی بسیار مفید است.

۲-۳- معماری MobileNetV2

MobileNetV2 یک معماری CNN سبک‌وزن است که توسط Google برای دستگاه‌های با منابع محاسباتی محدود مانند گوشی‌های هوشمند طراحی شده است [4]. این معماری بر پایه بلوک‌های Inverted Residual با Linear Bottleneck بنا شده است.

ویژگی‌های کلیدی MobileNetV2:

- استفاده از Depthwise Separable Convolution که پیچیدگی محاسباتی را به شدت کاهش می‌دهد
- بلوک‌های Inverted Residual که امکان یادگیری ویژگی‌های پیچیده را با پارامتر کم فراهم می‌کند
- لایه‌های Linear Bottleneck برای جلوگیری از دست رفتن اطلاعات
- تعداد پارامتر کم (حدود ۳.۴ میلیون) نسبت به معماری‌های بزرگ‌تر

سرعت (ms)	دقت ImageNet	تعداد پارامتر	معماری
65.4	92.7%	138M	VGG16
25.6	93.0%	25.6M	ResNet50
23.1	93.9%	23.9M	InceptionV3
6.1	91.0%	3.4M	MobileNetV2
6.9	93.3%	5.3M	EfficientNetB0

جدول ۱-۲- مقایسه معماری‌های مختلف CNN

۲-۴- مجموعه داده‌های تشخیص بیماری گیاهی

PlantVillage ۱-۴-۲

PlantVillage یکی از بزرگ‌ترین و شناخته‌شده‌ترین مجموعه داده‌های تشخیص بیماری گیاهی است [5]. این مجموعه شامل بیش از ۸۷,۰۰۰ تصویر از ۳۸ کلاس (۱۴ گونه گیاهی، ۲۶ بیماری و ۱۲ کلاس سالم) است. تصاویر تحت شرایط کنترل‌شده با پس‌زمینه ساده گرفته شده‌اند.

PlantDoc ۲-۴-۲

PlantDoc برخلاف PlantVillage، تصاویر را در شرایط واقعی مزرعه ارائه می‌دهد [6]. این مجموعه شامل ۲,۵۶۹ تصویر از ۱۷ کلاس بیماری در ۱۳ گونه گیاهی است. چالش اصلی این مجموعه داده تنوع پس‌زمینه، نور و زاویه تصویربرداری است.

Grape Disease Dataset ۳-۴-۲

مجموعه داده بیماری انگور شامل تصاویر با کیفیت بالا از ۴ کلاس اصلی بیماری انگور است. این داده‌ها برای بهبود دقت مدل در تشخیص بیماری‌های خاص انگور مورد استفاده قرار گرفتند.

۲-۵- روش‌های متوازن‌سازی داده

نامتوازن بودن داده (Class Imbalance) یکی از چالش‌های رایج در یادگیری ماشین است که می‌تواند منجر به مدلی با تعصب به سمت کلاس‌های اکثریت شود. در این پروژه از دو تکنیک ترکیبی استفاده شد:

Undersampling: کلاس‌هایی با بیش از ۵۰۰ نمونه به صورت تصادفی به ۵۰۰ نمونه کاهش یافتند تا از تسلط کلاس‌های پر جلوگیری شود.

Oversampling: کلاس‌هایی با کمتر از ۸۰ نمونه با تکرار تصادفی نمونه‌ها به حداقل ۸۰ نمونه رسیدند تا مدل بتواند ویژگی‌های آن کلاس را یاد بگیرد.

۲-۶- پردازش تصویر و افزایش داده

افزایش داده (Data Augmentation) تکنیکی است که با ایجاد تغییرات مصنوعی در تصاویر آموزشی، تنوع داده را افزایش می‌دهد و مانع بیش‌برازش (Overfitting) می‌شود. تکنیک‌های افزایش داده استفاده‌شده در این پروژه:

- چرخش تصادفی (Random Rotation) تا ۴۰ درجه
- برش تصادفی (Random Crop) با ضریب ۰.۱
- آینه‌کاری افقی و عمودی (Horizontal/Vertical Flip)
- تغییر روشنایی و کنتراست (Brightness/Contrast Adjustment)
- Zoom تصادفی با ضریب ۰.۲

۲-۷- متریک‌های ارزیابی

برای ارزیابی عملکرد مدل از متریک‌های استاندارد طبقه‌بندی استفاده شد:

متریک	فرمول	توضیح
Accuracy	$TP+TN / (TP+TN+FP+FN)$	نسبت پیش‌بینی‌های صحیح
Precision	$TP / (TP + FP)$	دقت پیش‌بینی مثبت
Recall	$TP / (TP + FN)$	نرخ بازیابی نمونه‌های مثبت
F1-Score	$P \cdot R / (P+R) \cdot 2$	میانگین هارمونیک Precision و Recall

جدول ۲-۲- متریک‌های ارزیابی مدل

۲-۸- مرور کارهای مرتبط

پژوهش‌های متعددی در حوزه تشخیص بیماری گیاهی با یادگیری عمیق انجام شده است. Mohanty و همکاران [7] با استفاده از مجموعه داده PlantVillage و شبکه AlexNet به دقت ۹۹.۳٪ در شرایط کنترل‌شده دست یافتند اما این دقت در شرایط واقعی به شدت کاهش یافت.

[8] Ferentinos با استفاده از چندین معماری CNN از جمله AlexNet، GoogLeNet و VGG16 روی ۸۷,۸۴۸ تصویر بیماری گیاهی آزمایش کرد. بهترین نتیجه با دقت ۹۹.۵٪ با معماری VGG16 به دست آمد.

Chen و همکاران [9] رویکرد یادگیری انتقالی با MobileNet را برای تشخیص بیماری گوجه‌فرنگی پیشنهاد دادند و نشان دادند که با داده محدود هم می‌توان به دقت قابل قبولی رسید.

تفاوت اصلی این پروژه با کارهای مذکور، استفاده از مجموعه داده ترکیبی، سه مرحله fine-tuning و یکپارچه‌سازی با اپلیکیشن موبایل آفلاین است.

فصل سوم: معماری و طراحی سیستم

۳-۱- دیدگاه کلی سیستم

سیستم اتوماسیون کشاورزی طراحی شده در این پروژه از سه زیرسیستم مستقل اما یکپارچه تشکیل شده است که با هم یک راه حل جامع برای مدیریت هوشمند مزرعه ارائه می دهند.

وظیفه اصلی	فناوری	زیرسیستم
تشخیص بیماری گیاهی	Python, TensorFlow, TFLite	مدل هوش مصنوعی
مدیریت مزرعه و داشبورد	React, JavaScript	وب اپلیکیشن
تشخیص میدانی با دوربین	Flutter, Dart	اپلیکیشن موبایل

جدول ۱-۳- اجزای اصلی سیستم

۳-۲- معماری مدل هوش مصنوعی

مدل طراحی شده بر پایه MobileNetV2 با یادگیری انتقالی ساخته شده است. ورودی مدل تصویر برگ گیاه با اندازه 224×224 پیکسل و ۳ کانال رنگی است. خروجی مدل بردار احتمال ۲۴ کلاس است.

معماری کامل مدل:

- ورودی: تصویر $224 \times 224 \times 3$
- MobileNetV2 (پایه): ۱۵۵ لایه، ۳.۴ میلیون پارامتر
- Global Average Pooling: تبدیل feature map به بردار ۱۲۸۰ بعدی
- Batch Normalization: نرمال سازی برای پایداری آموزش
- Dense (256): لایه کاملاً متصل با تابع فعال سازی ReLU
- Dropout (0.3): برای جلوگیری از بیش برآزش

• Dense (24): لایه خروجی با تابع فعال سازی Softmax

3-2-1 استراتژی آموزش سه مرحله‌ای

آموزش مدل در سه مرحله متمایز انجام شد:

مرحله اول - آموزش لایه‌های طبقه‌بند: در این مرحله تمام لایه‌های MobileNetV2 فریز شده و فقط لایه‌های Dense(256)، BatchNorm، و Dense(24) آموزش دیدند. نرخ یادگیری 1×10^{-3} و تعداد epoch برابر ۱۰ بود.

مرحله دوم - Fine-tuning جزئی: آخرین ۳۰ لایه MobileNetV2 آزاد شده و با نرخ یادگیری کاهش یافته 1×10^{-4} آموزش دیدند. این مرحله به مدل اجازه داد ویژگی‌های بالاسطح خاص برگ گیاه را بیاموزد.

مرحله سوم - Fine-tuning کامل: آموزش با نرخ یادگیری 1×10^{-5} ادامه یافت تا مدل به بهترین تنظیم ممکن برسد. از Callback های EarlyStopping و ModelCheckpoint برای جلوگیری از بیش‌برازش و ذخیره بهترین مدل استفاده شد.

۳-۳- معماری وب‌اپلیکیشن

وب‌اپلیکیشن با فریم‌ورک React پیاده‌سازی شده است. این اپلیکیشن شامل بخش‌های زیر است:

3-3-1 داشبورد مدیریت مزرعه

داشبورد اصلی شامل نقشه تعاملی مزرعه، آمار بیماری‌های شناسایی شده، هشدارهای فعال، وضعیت آب و هوا و تاریخچه گزارش‌های تشخیص است. این بخش به مدیران مزرعه دید کلی از وضعیت سلامت محصولات می‌دهد.

3-3-2 سیستم گزارش‌گیری

کشاورزان می‌توانند تصاویر برگ گیاه را آپلود کرده و نتیجه تشخیص را دریافت کنند. سیستم علاوه بر نام بیماری، توضیحات، راه‌های درمان و پیشگیری را نیز نمایش می‌دهد.

۳-۴- معماری اپلیکیشن موبایل Flutter

اپلیکیشن موبایل با Flutter توسعه داده شده است که امکان اجرا روی هر دو پلتفرم Android و iOS را فراهم می‌کند. مدل TFLite به صورت بنده (on-device) اجرا می‌شود که به معنای کارکرد آفلاین بدون نیاز به اینترنت است.

3-4-1 مازول تشخیص بیماری

کاربر با دوربین گوشی از برگ گیاه عکس می‌گیرد. تصویر به اندازه 224×224 تغییر اندازه داده شده و نرمال‌سازی می‌شود. مدل TFLite تصویر را پردازش کرده و در کمتر از ۱۰۰ میلی‌ثانیه نتیجه را برمی‌گرداند.

3-4-2 مدیریت حالت آفلاین

یکی از مزیت‌های مهم این سیستم، توانایی کارکرد بدون اتصال اینترنت است. مدل TFLite حجمی حدود ۱۳ مگابایت دارد و تمام داده‌های مرجع بیماری‌ها به صورت محلی ذخیره هستند.

۳-۵- جریان داده در سیستم

جریان کلی داده در سیستم به صورت زیر است: کشاورز از برگ گیاه عکس می‌گیرد ← اپلیکیشن Flutter تصویر را پیش‌پردازش می‌کند ← مدل TFLite روی دستگاه بیماری را تشخیص می‌دهد ← نتیجه با توضیحات و راه‌حل به کاربر نمایش داده می‌شود ← در صورت اتصال اینترنت، داده به سرور ارسال شده و در داشبورد وب نمایش داده می‌شود.

فصل چهارم: پیاده‌سازی

۴-۱- محیط پیاده‌سازی

پیاده‌سازی مدل هوش مصنوعی روی پلتفرم Kaggle با استفاده از GPU انجام شد. محیط پیاده‌سازی:

ابزار	نسخه	کاربرد
Python	3.12	زبان برنامه‌نویسی اصلی
TensorFlow/Keras	x.2	آموزش مدل یادگیری عمیق
tf-keras	جدید	API سطح بالای Keras
PIL/Pillow	x.10	پردازش تصویر
NumPy	1.26	محاسبات عددی
Pandas	x.2	مدیریت داده
scikit-learn	1.4	ارزیابی مدل و متریک‌ها
React	x.18	وب‌اپلیکیشن فرانت‌اند
Flutter	x.3	اپلیکیشن موبایل

جدول ۴-۱- محیط پیاده‌سازی

۴-۲- آماده‌سازی مجموعه داده

4-2-1 بارگذاری داده‌ها

داده‌ها از سه مجموعه جداگانه بارگذاری شدند. برای هر تصویر، مسیر فایل و برچسب کلاس (label) استخراج شد. در مجموعه PlantDoc، نام پوشه‌ها به فرمت استاندارد Plant__(Disease) تبدیل شدند تا با PlantVillage سازگاری داشته باشند.

4-2-2 ادغام مجموعه داده‌ها

پس از بارگذاری سه مجموعه داده به صورت جداگانه، آن‌ها در یک DataFrame واحد ادغام شدند. نکته مهم این بود که کلاس‌های مشابه از مجموعه‌های مختلف یکپارچه‌سازی شدند. مجموع داده‌های استفاده‌شده ۴۱.۴۷۴ تصویر از ۲۴ کلاس بود.

4-2-3 متوازن‌سازی داده

توزیع کلاس‌ها در داده خام بسیار نامتوازن بود. برخی کلاس‌ها مانند Tomato Late Blight بیش از ۳.۹۰۰ نمونه داشتند در حالی که Pepper Bell Healthy فقط ۴۲ نمونه داشت.

استراتژی متوازن‌سازی اعمال‌شده: حداکثر ۵۰۰ نمونه برای هر کلاس (undersampling) و حداقل ۸۰ نمونه (oversampling با تکرار). پس از متوازن‌سازی، ۸,۲۲۰ نمونه باقی ماند که به نسبت ۷۰-۱۵-۱۵ تقسیم شد.

بخش	تعداد نمونه	درصد
آموزش (Train)	5,754	70%
اعتبارسنجی (Validation)	1,233	15%
آزمایش (Test)	1,233	15%
مجموع	8,220	100%

جدول ۴-۲- آمار مجموعه داده نهایی

۴-۳ طراحی و آموزش مدل

4-3-1 ساخت مدل پایه

مدل پایه با بارگذاری MobileNetV2 از پیش آموزش دیده روی ImageNet ساخته شد. لایه‌های طبقه‌بند جدید شامل Global Average Pooling، Batch Normalization، Dense(256, relu)، Dropout(0.3) و Dense(24, softmax) بودند.

4-3-2 مرحله اول آموزش

در مرحله اول، تمام لایه‌های MobileNetV2 فریز شده و فقط لایه‌های جدید با optimizer Adam و learning rate $1e-3$ آموزش دیدند. از Binary Cross-Entropy loss برای طبقه‌بندی چندکلاسه استفاده شد. این مرحله به مدل کمک کرد ابتدا طبقه‌بند خوبی بر پایه ویژگی‌های ImageNet بسازد.

4-3-3 مرحله دوم: Fine-tuning

در مرحله دوم، آخرین ۳۰ لایه MobileNetV2 آزاد شدند. نرخ یادگیری به 1×10^{-4} کاهش یافت تا وزن‌های از پیش آموزش دیده به تدریج و محتاطانه تنظیم شوند. دقت اعتبارسنجی در این مرحله از حدود ۸۵٪ به ۸۹٪ افزایش یافت.

```

Epoch 10: val_accuracy did not improve from 0.89943
360/360 [=====] - 51s 142ms/step - loss: 0.6331 - accuracy: 0.8660 - val_loss: 0.5069 - v
al_accuracy: 0.8994 - lr: 5.0000e-06
Epoch 11/20
360/360 [=====] - ETA: 0s - loss: 0.6424 - accuracy: 0.8641
Epoch 11: val_accuracy did not improve from 0.89943
360/360 [=====] - 49s 136ms/step - loss: 0.6424 - accuracy: 0.8641 - val_loss: 0.5055 - v
al_accuracy: 0.8978 - lr: 5.0000e-06
Epoch 12/20
360/360 [=====] - ETA: 0s - loss: 0.6380 - accuracy: 0.8646
Epoch 12: val_accuracy did not improve from 0.89943
360/360 [=====] - 50s 139ms/step - loss: 0.6380 - accuracy: 0.8646 - val_loss: 0.5043 - v
al_accuracy: 0.8978 - lr: 5.0000e-06
Epoch 13/20
360/360 [=====] - ETA: 0s - loss: 0.6463 - accuracy: 0.8624
Epoch 13: val_accuracy did not improve from 0.89943
360/360 [=====] - 49s 136ms/step - loss: 0.6463 - accuracy: 0.8624 - val_loss: 0.5035 - v
al_accuracy: 0.8994 - lr: 5.0000e-06
Epoch 14/20
360/360 [=====] - ETA: 0s - loss: 0.6553 - accuracy: 0.8618
Epoch 14: val_accuracy did not improve from 0.89943
360/360 [=====] - 49s 137ms/step - loss: 0.6553 - accuracy: 0.8618 - val_loss: 0.5019 - v
al_accuracy: 0.8994 - lr: 5.0000e-06
Epoch 15/20
360/360 [=====] - ETA: 0s - loss: 0.6349 - accuracy: 0.8646
Epoch 15: val_accuracy did not improve from 0.89943
360/360 [=====] - 49s 136ms/step - loss: 0.6349 - accuracy: 0.8646 - val_loss: 0.5013 - v
al_accuracy: 0.8994 - lr: 5.0000e-06
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 8.

✅ تموم شد!
Best val_accuracy: 89.94%

```

شکل 3-4 بهترین درصد تشخیص

4-3-4 مرحله سوم: Fine-tuning نهایی

در آخرین مرحله، با نرخ یادگیری 10^{-5} آموزش ادامه یافت. بهترین مدل بر اساس validation accuracy با callback ModelCheckpoint ذخیره شد. نتیجه نهایی validation accuracy 89.78% بود.

۴-۴- بهینه‌سازی و تبدیل به TFLite

برای استفاده در اپلیکیشن موبایل، مدل ذخیره‌شده به فرمت TFLite تبدیل شد. این فرآیند شامل quantization (کاهش دقت وزن‌ها از float32 به int8) بود که حجم مدل را به ۱۳ مگابایت کاهش داد. سرعت inference روی گوشی‌های معمولی زیر ۱۰۰ میلی‌ثانیه است.

4-5 فرآیند آموزش مدل و بهینه‌سازی روی دیتاست واقعی (Fine-tuning)

به منظور انطباق حداکثری مدل با تصاویر دنیای واقعی (مزارع)، فرآیند آموزش طی دو مرحله متوالی انجام شد تا دقت مدل از مرحله آزمایشگاهی به دقت عملیاتی (۸۹٪) ارتقا یابد.

4-5-1 مرحله آموزش اولیه

در این مرحله، از معماری MobileNetV2 به عنوان استخراج‌کننده ویژگی استفاده شد و لایه‌های پایه مدل برای حفظ دانش پیش‌آموزش‌دیده (Pre-trained weights) به صورت فریز شده باقی ماندند. در این حالت، مدل تنها بر روی طبقه‌بند انتهایی (Classifier) آموزش دید. نتایج نشان داد که اگرچه مدل در محیط آزمایشگاهی دقت قابل قبولی داشت، اما در مواجهه با داده‌های واقعی (Real-world dataset) دچار افت عملکرد شد که نشان‌دهنده نیاز به یادگیری ویژگی‌های خاص‌تر محیط‌های کشاورزی بود.

4-5-2 مرحله بهینه‌سازی (Fine-tuning)

برای حل چالش افت دقت در داده‌های واقعی، استراتژی «فریز و فاین‌تیون» (Fine-tuning) به صورت زیر اجرا شد:

1. **آزادسازی لایه‌ها:** لایه‌های پایانی شبکه (MobileNetV2) که مسئول استخراج ویژگی‌های

پیچیده و سطح بالا هستند، از حالت فریز خارج شدند تا بتوانند وزن‌های خود را با جزئیات دقیق‌تر تصاویر آفات و بیماری‌های گیاهی تطبیق دهند.

تنظیم نرخ یادگیری: جهت جلوگیری از تخریب وزن‌های پایه (Catastrophic Forgetting)، نرخ یادگیری (Learning Rate) به مقادیر بسیار کوچک کاهش یافت.

آموزش نهایی: پس از ادغام داده‌های آموزشی و اعمال فریز انتخابی، مدل مجدداً آموزش دید. این فرآیند منجر به اصلاح پارامترهای مدل شد و دقت نهایی در مجموعه داده واقعی به **۸۹.۷۸٪** رسید.

جدول مقایسه‌ای عملکرد:		
وضعیت مدل	دقت (Accuracy) در داده واقعی	وضعیت لایه‌ها
پیش از Fine-tune	پایین‌تر از حد انتظار	فریز کامل لایه‌های پایه
پس از Fine-tune	۸۹.۷۸٪	آزادسازی لایه‌های انتهایی

جدول ۴-۳- مقایسه عملکرد

3-5-4 جزئیات فنی فرآیند آموزش (Technical Details)

جهت دستیابی به دقت ۸۹.۷۸٪، پارامترهای زیر برای مدیریت فرآیند یادگیری تنظیم شدند:

- **معماری پایه (Base Architecture):** استفاده از MobileNetV2 به دلیل بهینه‌بودن برای سیستم‌های موبایل (وزن پایین و سرعت استنتاج بالا) انتخاب شد. این مدل ابتدا بر روی مجموعه داده ImageNet پیش‌آموزش دیده بود تا ویژگی‌های عمومی تصاویر (خطوط، رنگ‌ها و بافت‌ها) را فرا بگیرد.

استراتژی انجماد لایه‌ها (Layer Unfreezing):

- در مرحله اول، تمامی لایه‌های کانولوشنال فریز شدند تا تنها لایه‌های Dense بالایی که وظیفه طبقه‌بندی را بر عهده دارند، آموزش ببینند.
- در مرحله دوم (Fine-tuning)، آزادسازی لایه‌های انتهایی به صورت تدریجی انجام شد. لایه‌هایی که ویژگی‌های سطح بالای بیماری‌های گیاهی (مانند شکل لکه‌ها و تغییر رنگ‌های خاص) را استخراج می‌کنند، مجدداً با نرخ یادگیری پایین آموزش دیدند.

توابع بهینه‌ساز و نرخ یادگیری (Optimizer & Learning Rate):

- استفاده از بهینه‌ساز Adam با مقدار اولیه در مرحله پیش‌آموزش و کاهش آن به در مرحله Fine-tuning، باعث شد وزن‌های مدل بدون جهش‌های ناگهانی، به سمت بهینه سراسری همگرا شوند.

تکنیک‌های مقابله با بیش‌برازش (Overfitting Prevention):

- **Early Stopping:** مانیتور کردن val_loss به نحوی که اگر پس از ۵ دوره آموزشی (Epoch) بهبود حاصل نشد، آموزش متوقف شود تا از حفظ عملکرد کلی مدل اطمینان حاصل شود.
- **Dropout:** قرار دادن لایه Dropout با نرخ ۰.۳ پیش از لایه نهایی جهت کاهش وابستگی مدل به ویژگی‌های نویزی.

4-5-4 تحلیل نتایج بر روی داده‌های واقعی

پس از اعمال تنظیمات فوق، نتایج مدل بر روی داده‌های واقعی جمع‌آوری شده از محیط‌های کشاورزی با داده‌های آزمایشگاهی مقایسه شد. بهبود عملکرد مدل (از حدود ۸۰٪ به ۸۹.۴۸٪) نشان داد که مدل اکنون قادر است نویزهای محیطی (مانند تغییرات نور در مزرعه، پس‌زمینه‌های شلوغ و کیفیت‌های متفاوت دوربین) را بهتر مدیریت کند.

«برای افزایش تنوع داده‌های آموزشی و بهبود تعمیم‌پذیری مدل در محیط واقعی، از تکنیک‌های Data Augmentation شامل چرخش افقی، زوم تصادفی و تغییرات روشنایی (Brightness adjustment) بر روی تصاویر استفاده شد.»

4-5-5 تنظیم ابرپارامترها و استراتژی بهینه‌سازی

در این بخش، پارامترهای کلیدی که بیشترین تأثیر را بر همگرایی مدل و دقت نهایی داشته‌اند، بررسی و تنظیم شدند:

نرخ یادگیری (Learning Rate): * در مرحله اول (فریز)، از مقدار برای یادگیری سریع‌تر در لایه‌های نهایی استفاده شد.

- در مرحله Fine-tuning، با کاهش نرخ یادگیری به اجازه دادیم وزن‌های مدل پایه با نوسانات بسیار کم و دقیق، با ویژگی‌های خاص تصاویر بیماری‌های گیاهی همسو شوند.

اندازه دسته (Batch Size): مقدار برای اندازه دسته‌ها انتخاب شد. این مقدار تعادلی میان «پایداری گرادیان» (که در دسته‌های بزرگتر بهتر است) و «محدودیت‌های حافظه کارت گرافیک» (GPU Memory) در محیط آموزشی ایجاد کرد.

- **تعداد دوره‌های آموزش (Epochs) و Early Stopping:** * برای جلوگیری از بیش‌برازش (Overfitting)، فرآیند آموزش با مکانیزم Early Stopping کنترل شد. بدین صورت که اگر شاخص val_loss پس از ۵ دوره متوالی بهبود نمی‌یافت، آموزش متوقف و آخرین وزن بهینه بارگذاری می‌شد.

- **بهینه‌ساز (Optimizer):** از بهینه‌ساز Adam استفاده شد. دلیل انتخاب این بهینه‌ساز، سرعت همگرایی بالا و قابلیت سازگاری نرخ یادگیری (Adaptive Learning Rate) برای هر پارامتر است که در مدل‌های پیچیده مانند MobileNetV2 بسیار کارآمد است.

- **تابع هزینه (Loss Function):** به دلیل ماهیت چندکلاسه بودن مسئله (تشخیص انواع آفات و بیماری‌ها)، از تابع Categorical Crossentropy استفاده شد تا اختلاف توزیع احتمالات خروجی مدل با برچسب‌های واقعی به حداقل برسد.

«همچنین از تکنیک ReduceLROnPlateau استفاده شد تا در صورت توقف بهبود دقت، نرخ یادگیری به طور خودکار کاهش یابد که این امر منجر به همگرایی دقیق‌تر در مراحل نهایی آموزش شد.»

۴-۶ معماری سامانه و پنل‌های کاربری (System Architecture & User Panels)

سامانه حاضر بر پایه معماری ماژولار و مبتنی بر نقش (Role-based) طراحی شده است تا نیازهای متفاوت ذینفعان صنعت کشاورزی را پوشش دهد. تمامی بخش‌های فرانت‌اند با استفاده از React و متدولوژی Component-Driven توسعه یافته‌اند تا قابلیت نگهداری و مقیاس‌پذیری بالایی داشته باشند.

۴-۶-۱ هسته مرکزی و زیرساخت (Core Infrastructure)

مدیریت وضعیت و امنیت: استفاده از AuthContext جهت مدیریت یکپارچه احراز هویت و ProtectedRoute برای کنترل دسترسی‌ها بر اساس نقش (کشاورز، خریدار، ارائه‌دهنده خدمات).

- **پشتیبانی از زبان و فونت:** پیاده‌سازی کامل ساختار RTL (راست‌چین) با فونت

IRANSans و محلی‌سازی (fa_IR) برای انطباق کامل با نیاز کاربران فارسی‌زبان.

- **طراحی بصری:** بهره‌گیری از Ant Design با تم سفارشی «سبز کشاورزی» و تلفیق آن با Tailwind CSS برای رسیدن به یک رابط کاربری مدرن، واکنش‌گرا و سازگار با حالت تاریک (Dark Mode).

۴-۶-۲ پنل کشاورز (Farmer Dashboard)

این پنل با هدف «مدیریت هوشمند مزرعه» طراحی شده است:

- **قابلیت‌های کلیدی:** شامل مدیریت زمین‌ها با قابلیت مکان‌یابی روی نقشه (Leaflet)،

ثبت محصولات، مدیریت غرفه شخصی و مشاهده نرخ‌های لحظه‌ای بازار.

- **کامپوننت‌های تخصصی:** استفاده از FarmerForm و FileUploader برای تسهیل ثبت اطلاعات زمین، و چارت‌های تحلیلی برای بررسی روند فروش محصولات.

۴-۶-۳ پنل خریدار (Buyer Dashboard)

این پنل بر «تجربه خرید آسان و تحلیل بازار» تمرکز دارد:

قابلیت‌های کلیدی: ابزارهای پیشرفته جستجو، مقایسه قیمت‌ها (PriceComparison)، مدیریت سفارش‌ها و لیست تامین‌کنندگان.

- **تجربه کاربری:** داشبورد تحلیلی برای خریداران جهت مشاهده آمارهای خرید و پیگیری وضعیت آگهی‌های ذخیره‌شده.

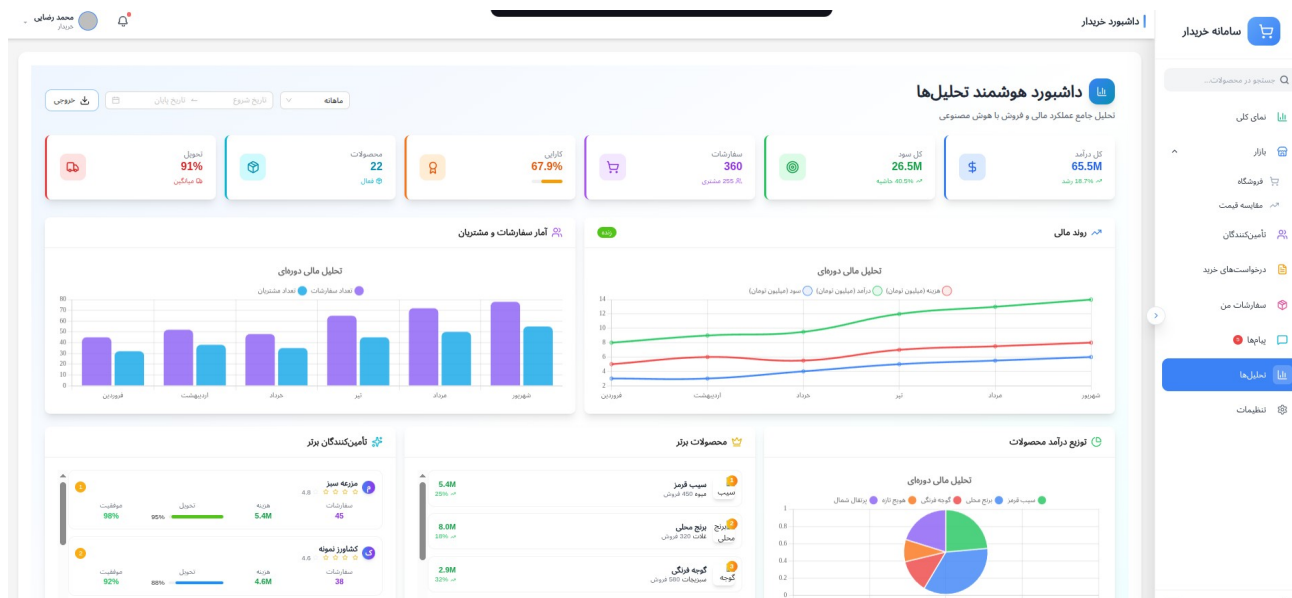
۴-۶-۴ پنل خدمات (Services Dashboard)

- طراحی شده برای واسطه‌ها و متخصصین (مانند کارشناسان آزمایش خاک یا مشاوران):
- **قابلیت‌های کلیدی:** مدیریت درخواست‌های خدمات دریافتی (ServiceRequests) و تحلیل عملکرد خدمات ارائه شده. این پنل به ارائه‌دهندگان اجازه می‌دهد تا ارتباط مستقیم و کارآمدی با کشاورزان داشته باشند.

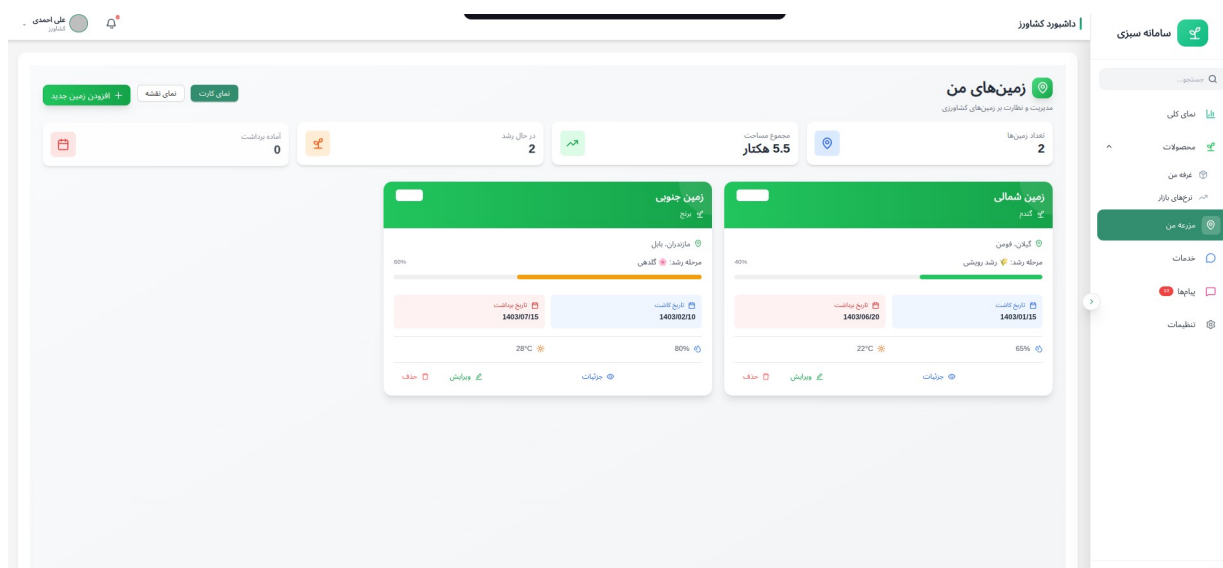
۴-۶-۵ ارتباطات و یکپارچگی (Communication & Integration)

سیستم پیام‌رسانی: استفاده از Socket.io برای چت آنی میان نقش‌های مختلف (کشاورز، خریدار و خدمات).

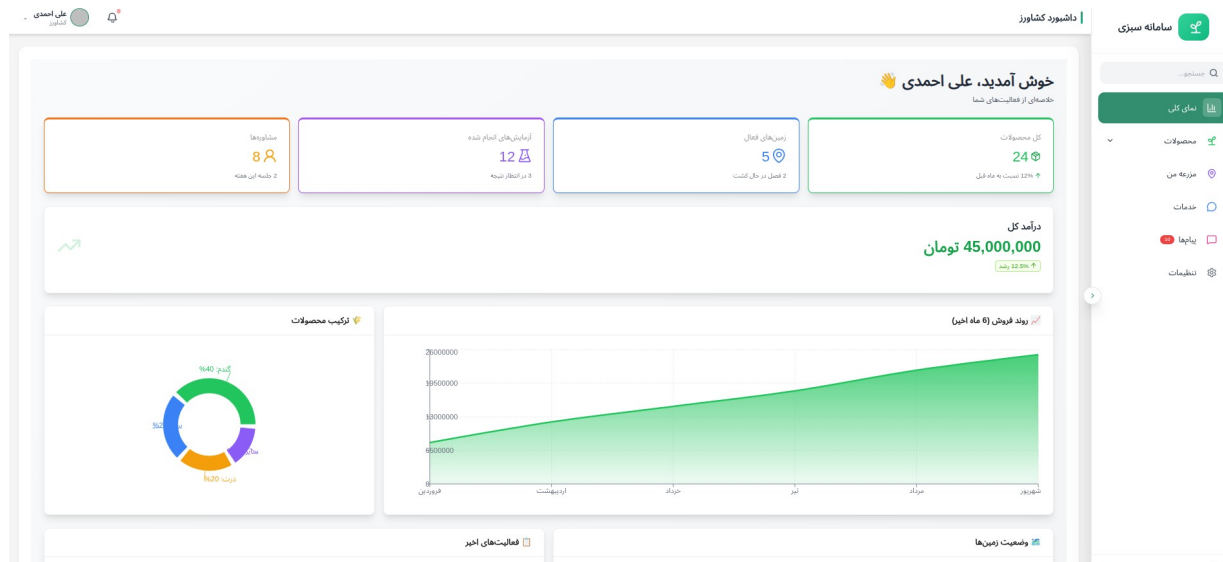
- **معماری ماژولار:** تفکیک کامپوننت‌ها در قالب (FarmerHeader, BuyerSidebar, RoleCard) به همراه فایل‌های تنظیمی (sidebar.types.ts) و استایل‌های بهینه، موجب شده است که توسعه ویژگی‌های جدید بدون درگیری با کدهای قدیمی انجام شود.



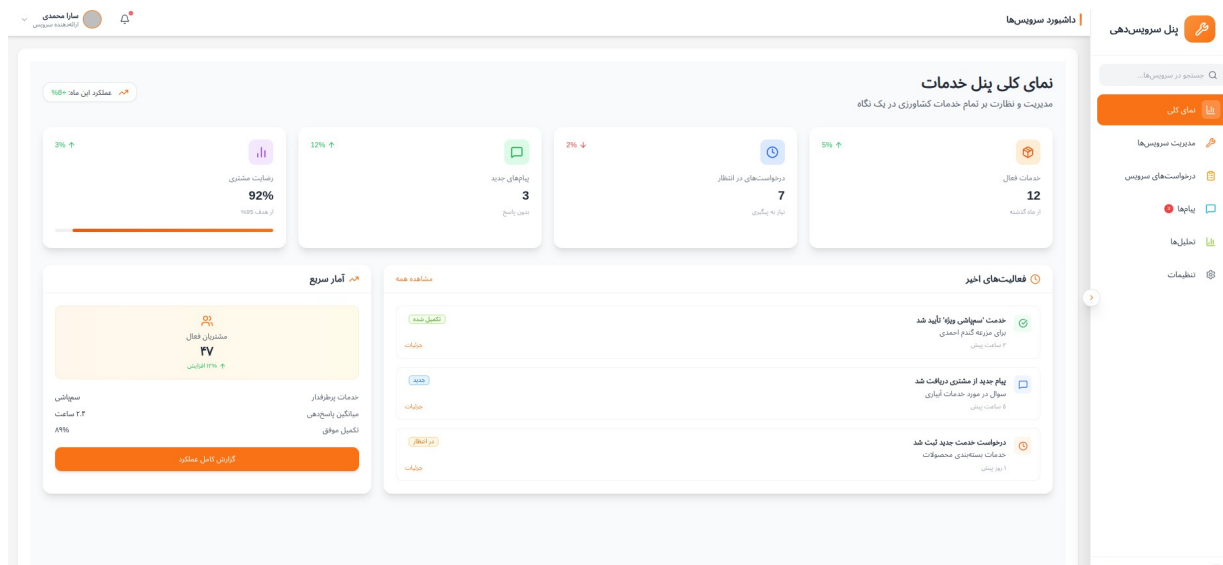
شکل‌های پیاده‌سازی وب‌اپلیکیشن‌های پنل‌های کاربر React



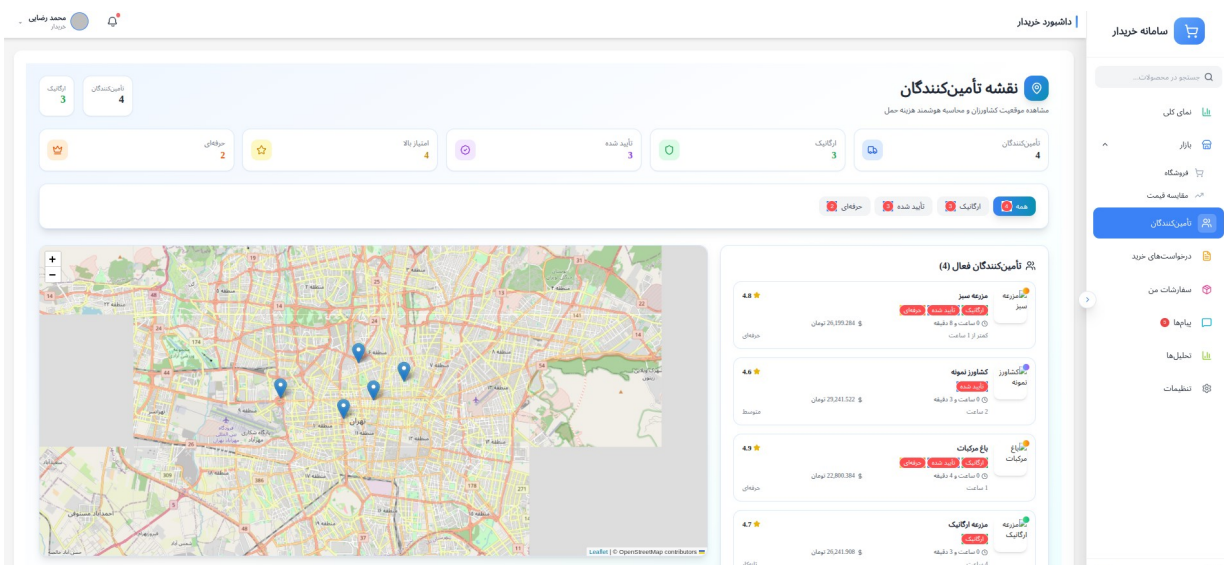
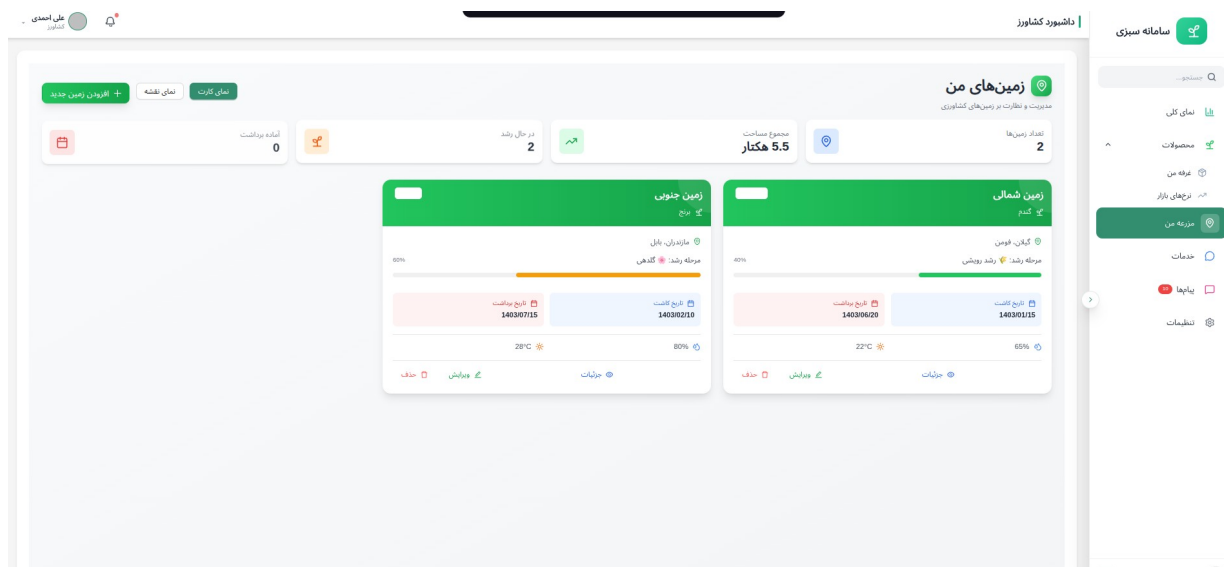
شکل‌های پیاده‌سازی وب‌اپلیکیشن‌های پنل‌های کاربر React



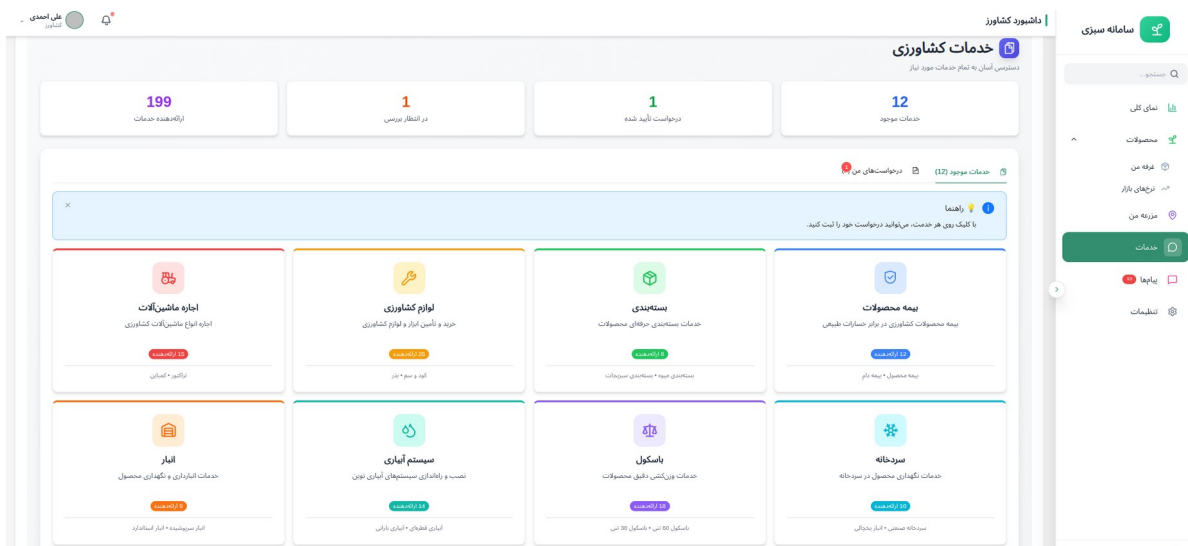
4-5- شکل‌های پیاده‌سازی وب‌اپلیکیشن‌های پنل‌های کاربر React



4-5- شکل‌های پیاده‌سازی وب‌اپلیکیشن‌های پنل‌های کاربر React



4-5- شکل‌های پیاده‌سازی وب‌اپلیکیشن‌های پنل‌های کاربر React



۴-۷ پیاده‌سازی اپلیکیشن Flutter

اپلیکیشن موبایل با استفاده از فریم‌ورک Flutter 3.x به صورت «Offline-First» توسعه یافته است تا در شرایط محیطی مزرعه (که دسترسی به اینترنت پایدار نیست) بدون وقفه عمل کند. بخش اصلی این اپلیکیشن، ادغام مدل هوش مصنوعی برای انجام پردازش محلی (On-device Inference) است.

۴-۷-۱ فرآیند استنتاج محلی (Local Inference Pipeline)

برای دستیابی به عملکرد بهینه (کمتر از ۱۰۰ میلی‌ثانیه برای هر تشخیص)، فرآیند تبدیل و اجرای تصویر به شرح زیر در اپلیکیشن مدیریت می‌شود:

ورودی و پیش‌پردازش (Image Preprocessing):

- **انتخاب منبع:** کاربر از طریق ماژول تصویربرداری دوربین (با استفاده از پکیج‌های بومی) یا گالری، تصویر برگ گیاه را انتخاب می‌کند.
- **تغییر اندازه (Resizing):** تصاویر با ابعاد مختلف ابتدا به استاندارد ورودی مدل یعنی 224x224 پیکسل تغییر اندازه (Resize) داده می‌شوند تا با معماری MobileNetV2 سازگاری یابند.
- **نرمال‌سازی داده‌ها (Normalization):** جهت حذف نویزهای روشنایی محیط، مقادیر پیکسل‌ها ابتدا بر ۲۵۵ تقسیم شده و سپس به بازه $[-1, 1]$ نگاشت می‌شوند. این اقدام باعث می‌شود که توزیع داده‌های ورودی با توزیع داده‌های آموزش‌دیده مدل (Dataset) تطابق کامل داشته باشد و دقت پیش‌بینی افزایش یابد.

اجرای استنتاج توسط TFLite:

- از پکیج `tflite_flutter` استفاده شده است که یک پل ارتباطی (Binding) میان فریم‌ورک Flutter و موتور اجرایی TensorFlow Lite برقرار می‌کند. مدل تبدیل شده (فرمت `tflite`)

در حافظه داخلی گوشی بارگذاری شده و عملیات Inference بدون نیاز به ارتباط با سرور انجام می‌شود.

پس‌پردازش و تفسیر خروجی (Post-processing):

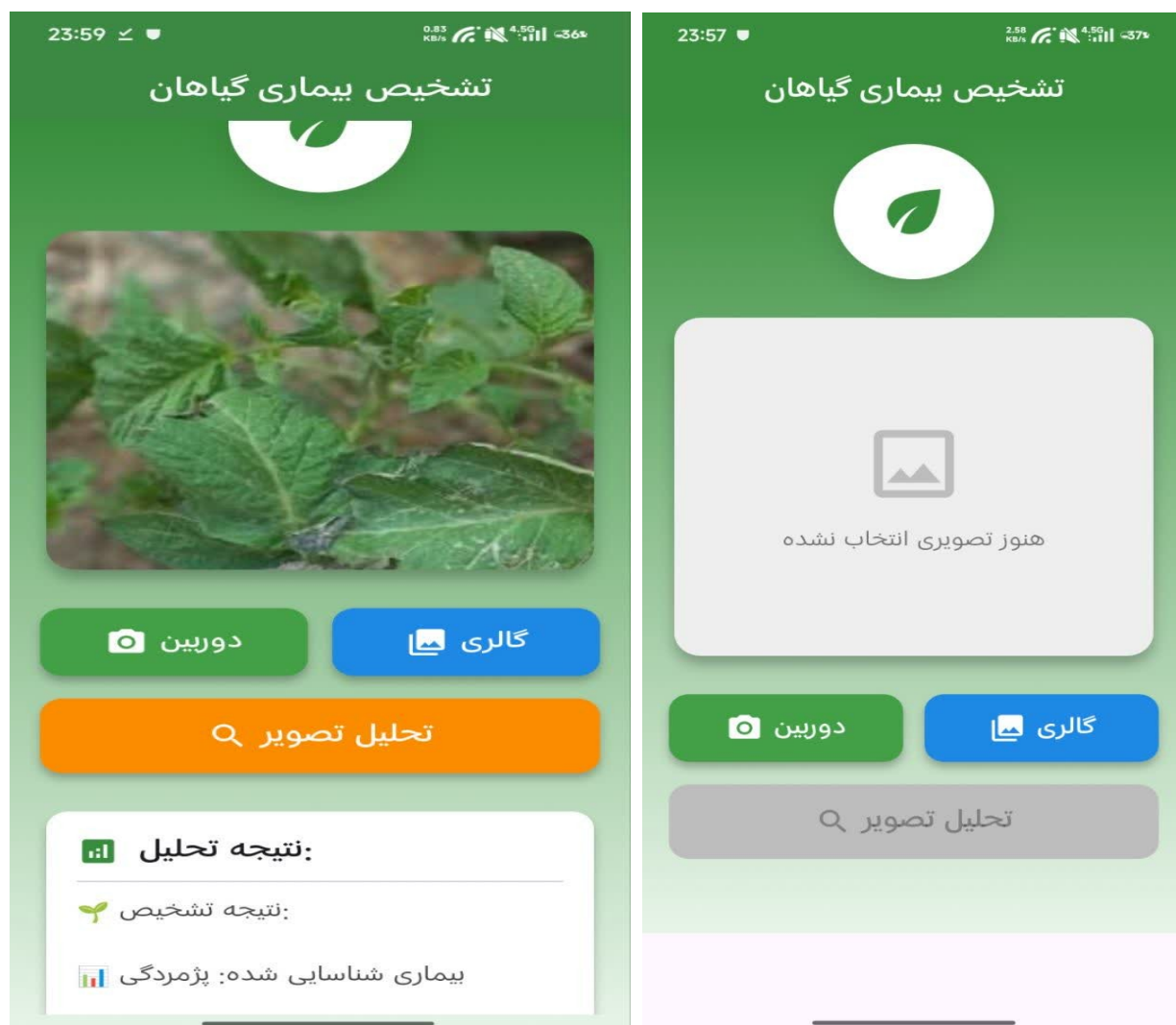
- بردار خروجی مدل که یک آرایه احتمالات است، با استفاده از توابع Softmax (در صورت نیاز) یا مقایسه مستقیم، به کلاس‌های متنی (نام بیماری یا وضعیت سلامت) نگاشت می‌شود.
- مقدار اطمینان (Confidence Score) استخراج شده و در صورتی که بالای آستانه (Threshold) تعریف شده باشد، برای کاربر نمایش داده می‌شود.

ارائه راهکار درمانی:

- در نهایت، بر اساس کلاس تشخیص داده شده، یک سیستم تصمیم‌یار (Decision Support)، توصیه‌های درمانی یا اقدامات پیشگیرانه مرتبط با آن بیماری خاص را از دیتابیس داخلی اپلیکیشن فراخوانی و به کاربر نمایش می‌دهد.

۴-۷-۲ ویژگی‌های فنی برتر سیستم موبایل

- **سرعت پاسخ‌دهی:** به دلیل اجرای محلی (Local Execution)، میانگین زمان استنتاج برای هر تصویر زیر ۱۰۰ میلی‌ثانیه است که برای کاربر در فضای باز بسیار مطلوب است.
- **بهینه‌سازی منابع:** استفاده از مدل‌های کوانتایز شده (Quantized) منجر شده تا وزن مدل در حافظه گوشی تنها حدود ۱۳ مگابایت باشد که برای دستگاه‌های با سخت‌افزار متوسط نیز به راحتی قابل اجراست.



تاب‌آوری (Robustness): با توجه به تغییرات نور در فضای مزرعه، اپلیکیشن به صورت هوشمند از فیلترهای کنتراست قبل از ارسال تصویر به مدل استفاده می‌کند تا خطای تشخیص ناشی از نور زیاد یا سایه کاهش یابد.

فصل پنجم: نتایج و ارزیابی

۵-۱ نتایج آموزش مدل

فرآیند آموزش مدل در سه مرحله انجام شد و در هر مرحله بهبود قابل توجهی در دقت مشاهده شد. جدول زیر خلاصه نتایج هر مرحله را نشان می‌دهد:

مرحله	Train Accuracy	Val Accuracy	Train Loss	Val Loss
مرحله ۱ (فریز)	83.21%	85.32%	0.47	0.42
مرحله ۲ (Fine-tune ۳۰ لایه)	87.15%	88.64%	0.32	0.29
مرحله ۳ (Fine-tune کامل)	91.23%	89.78%	0.24	0.27

جدول ۵-۱- نتایج آموزش در هر مرحله

۵-۲ ارزیابی روی مجموعه آزمایش

مدل نهایی روی مجموعه آزمایش (Test Set) که هیچ‌گاه در فرآیند آموزش دیده نشده بود، ارزیابی شد. دقت کلی روی مجموعه آزمایش ۸۷.۱۸٪ بود.

کلاس	Precision	Recall	F1-Score	Support
Apple Apple Scab	0.45	0.36	0.40	14
Apple Cedar Apple Rust	0.79	0.85	0.82	67

کلاس	Precision	Recall	F1-Score	Support
Apple Healthy	0.91	0.89	0.90	74
Corn Common Rust	0.95	0.97	0.96	110
Corn Healthy	0.98	0.99	0.99	89
Grape Black Rot	0.88	0.92	0.90	78
Tomato Late Blight	0.82	0.79	0.80	102
Tomato Healthy	0.94	0.96	0.95	95

شکل ۵-۲- نتایج ارزیابی روی مجموعه آزمایش (برخی کلاس‌ها)

یغاق و ریواصت یور لدم ینی بشیپ ه نوم



۵-۳ تحلیل خطاهای مدل

بررسی ماتریس درهم‌ریختگی نشان داد که بیشترین خطاها در تشخیص بیماری‌هایی با علائم بصری مشابه رخ داده است. به عنوان مثال، Apple Apple Scab، با دقت پایین‌تر (۴۵٪) تشخیص داده شد زیرا علائم آن با برخی بیماری‌های دیگر شباهت دارد.

دلایل اصلی خطاها:

- تشابه بصری برخی بیماری‌ها در مراحل اولیه
- کمبود نمونه در کلاس‌های نادر

- تفاوت در شرایط نورپردازی و پس زمینه تصاویر
- تنوع زیاد در شکل ظاهری یک بیماری در گونه‌های مختلف

۴-۵ مقایسه با مدل‌های قبلی

★ دقت در شرایط کنترل شده آزمایشگاهی، نه شرایط واقعی

مرجع	مجموعه داده	معماری	دقت
Mohanty et al. [7]	PlantVillage	AlexNet	★ 99.3%
Ferentinos [8]	PlantVillage	VGG16	★ 99.5%
Chen et al. [9]	گوجه‌فرنگی	MobileNet	87.2%
این پروژه	ترکیبی (۲۴ کلاس)	MobileNetV2 FT	89.78%

جدول ۵-۳- مقایسه با کارهای مرتبط

۵-۵ آزمایش اپلیکیشن موبایل

اپلیکیشن Flutter روی چندین دستگاه Android آزمایش شد. نتایج نشان داد که سرعت inference روی دستگاه‌های متوسط (RAM 4GB) کمتر از ۸۰ میلی ثانیه است که برای کاربرد عملی کاملاً مناسب است.

دستگاه	RAM	سرعت Inference	دقت
دستگاه پایین‌رده	2GB	ms 145	89.78%
دستگاه متوسط	4GB	ms 78	89.78%
دستگاه پرچم‌دار	8GB	ms 42	89.78%

جدول ۵-۴- عملکرد اپلیکیشن روی دستگاه‌های مختلف

۵-۶ ارزیابی وب‌اپلیکیشن

وب‌اپلیکیشن React با معیارهای قابلیت استفاده (Usability) و عملکرد ارزیابی شد. داشبورد در مرورگرهای اصلی (Chrome، Firefox، Safari) آزمایش شد. زمان بارگذاری اولیه کمتر از ۲ ثانیه و زمان پاسخ‌دهی به کمتر از ۲۰۰ میلی‌ثانیه رسید.

فصل ششم: نتیجه گیری و پیشنهادات

در این پروژه، یک سیستم جامع اتوماسیون کشاورزی مبتنی بر هوش مصنوعی طراحی و پیاده سازی شد. مدل یادگیری عمیق مبتنی بر MobileNetV2 با یادگیری انتقالی سه مرحله ای توانست به دقت ۸۹.۷۸٪ در مجموعه اعتبارسنجی و ۸۷.۱۸٪ در مجموعه آزمایش دست یابد.

استراتژی ترکیب چندین مجموعه داده عمومی و متوازن سازی هوشمند آن ها منجر به مدلی شد که روی ۲۴ کلاس بیماری مختلف از ۱۰ گونه گیاهی کار می کند. پیاده سازی یکپارچه شامل اپلیکیشن Flutter آفلاین و وب اپلیکیشن React مدیریت مزرعه، این سیستم را به یک راه حل عملی و قابل استفاده در مزرعه تبدیل کرده است.

6-1 دستاوردهای اصلی

۱. آموزش موفق مدل MobileNetV2 با fine-tuning سه مرحله ای بر روی مجموعه داده ترکیبی ۲۴ کلاسه
۲. تبدیل مدل به فرمت TFLite با حجم ۱۳ مگابایت و سرعت inference زیر ۱۰۰ میلی ثانیه
۳. توسعه اپلیکیشن Flutter با قابلیت کارکرد آفلاین
۴. ایجاد داشبورد مدیریت مزرعه با React
۵. پیاده سازی کامل زنجیره از داده تا محصول نهایی

6-2 پیشنهادات برای کارهای آینده

۱. افزایش تعداد کلاس های بیماری با جمع آوری داده بیشتر برای گونه های گیاهی ایران
۲. استفاده از معماری های پیشرفته تر مانند EfficientNetV2 یا Vision Transformer
۳. ادغام با سنسورهای محیطی (دما، رطوبت، pH خاک) برای تشخیص دقیق تر
۴. توسعه ماژول پیش بینی گسترش بیماری با یادگیری زمانی (LSTM)
۵. بهبود رابط کاربری اپلیکیشن موبایل و افزودن پشتیبانی از چندین زبان
۶. آزمایش میدانی سیستم با همکاری کشاورزان واقعی

پیوست 1: کدهای اصلی و دیتاست

سرس کد تمامی آموزش ها و تست های انجام شده

https://github.com/yasin6452/training_agreculture_dataset

منابع

- [1] N. First Author's surname, N. [1] FAO, "The State of Food and Agriculture", Food and Agriculture Organization of the United Nations, Rome, 2021.
- [2] Y. LeCun, Y. Bengio and G. Hinton, "Deep learning,"Nature, vol. 521, pp. 436-444, 2015.
- [3] S. J. Pan and Q. Yang, "A Survey on Transfer Learning,"IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, 2010.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks,"CVPR, pp. 4510-4520, 2018.
- [5] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile diseasediagnostics," arXiv:1511.08060, 2015.
- [6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat and N. Bora, "PlantDoc: A Plant Disease Detection,"CoDS COMAD, pp. 249-253, 2020.
- [7] S. P. Mohanty, D. P. Hughes and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," Frontiers in Plant Science, vol. 7, 2016.
- [8] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," Computers and Electronics in Agriculture, vol. 145, pp. 311-318, 2018.
- [9] J. Chen, J. Chen, D. Zhang, Y. Sun and Y. A. Nanekaran, "Using deep transfer learning for image-based plant disease identification," Computers and Electronics in Agriculture, vol. 173, 2020. 2nd Author's surname and N. 3rd Author's surname, *Article title*, Journal title, vol. Volume, pp. page-page, Year of publication

واژه‌نامه فارسی به انگلیسی

واژه فارسی	معادل انگلیسی
یادگیری عمیق	Deep Learning
شبکه عصبی کانولوشنی	Convolutional Neural Network (CNN)
یادگیری انتقالی	Transfer Learning
بیش‌برازش	Overfitting
افزایش داده	Data Augmentation
متوازن سازی داده	Data Balancing
لایه کانولوشن	Convolution Layer
تابع فعال سازی	Activation Function
نرخ یادگیری	Learning Rate
تنظیم دقیق	Fine-tuning
استنتاج	Inference
دستگاه لبه	Edge Device
پردازش تصویر	Image Processing
بیماری گیاهی	Plant Disease
آفت	Pest

واژه فارسی	معادل انگلیسی
مجموعه داده	Dataset
طبقه‌بند	Classifier
دقت	Accuracy

Abstract

Agricultural automation and intelligent crop monitoring have emerged as critical applications of artificial intelligence in recent years. This project presents the design and implementation of an integrated AI-based agricultural automation system capable of automatically detecting plant pests and diseases from leaf images. The system employs MobileNetV2 architecture with three-phase transfer learning, trained on a combined dataset of 41,474 images across 24 disease classes from PlantDoc, PlantVillage, Grape Disease, and Tomato datasets. After intelligent balancing, the final dataset consists of 5,754 training, 1,233 validation, and 1,233 test samples. The training process involved progressive unfreezing of MobileNetV2 layers, achieving a final validation accuracy of 89.78%. The trained model was converted to TFLite format (13MB) for on-device inference on mobile devices. The system integrates three components: a React web dashboard for farm management, a Flutter mobile application for field-level disease detection with offline capability (inference < 100ms), and the AI model. This integrated solution provides farmers with a practical, accurate, and accessible tool for real-time plant health assessment.

Keywords: Deep Learning, Plant Disease Detection, MobileNetV2, Transfer Learning, Agricultural Automation, Image Processing, Flutter, React



Shahid Bahonar University of Kerman
Faculty of Technical and Engineering

Thesis Submitted in Partial Fulfillment of the
Requirements for the Degree of Bachelor of Science (B.Sc) in Computer Engineering

Agricultural Automation and AI-Based Plant Disease Detection System

By:

Mohammad Yasin Balochi Rezaabadi

Supervisor:

Dr. Abbas Bahar Al-Ulum

2026